EAC 美業集團控制型平台系統架構討論版 v1

0. 文件定位

本文件用途：提供業主討論「要做什麼、先做什麼、系統怎麼長、哪些控制權一定要掌握」。

本文件不是畫面稿，也不是最終合約，而是可作為後續：

- 系統規劃依據
- 開發商詢價依據
- 內部決策與優先級討論依據

---

1. 專案摘要

1.1 專案一句話描述

打造一套以**數據主權、分潤透明、VIP 控制、加盟擴張**為核心的美業集團營運平台，讓總部能掌握客戶、交易、分潤與跨層級營運控制權。

1.2 核心目標

1. 客戶資料、交易資料、服務資料都留在集團自己的系統中。
2. 支援總部、賦能層、經銷商、加盟店、服務人員、顧客/VIP 的多層級運作。
3. 讓預約、派工、付款、服務、回報、對帳形成完整閉環。
4. 讓分潤、VIP、合約、產品、課程等商業規則逐步數位化。
5. 讓未來加盟擴張、多據點管理、跨區複製不需要重做底層架構。

1.3 系統定位

這不是單純預約系統，也不是單店後台。

它的正確定位是：
美業集團控制型平台（Control Platform）

本質上包含三個層次：

- 營運系統：預約、排班、付款、服務紀錄
- 平台系統：多角色、多據點、多層級資料隔離

- 控制系統：客戶主權、VIP 主權、流量分配、分潤規則

1.4 規模假設（可被推翻）

- 初期：1 個總部、1 間旗艦店、5–20 位服務人員
- 中期：3–10 間加盟店、10–50 位服務人員
- 後期：多經銷商、多加盟店、跨區域營運
- 前期以 Web 後台 + 顧客端 PWA + 服務人員端 PWA 為主
- 金流以第三方金流為主，不自建支付清算能力

---

2. 本次刻意不做（Out of Scope）

1. 原生 iOS / Android App
2. 即時通訊聊天系統
3. 複雜 AI 推薦與即時最佳化排班
4. POS 硬體、IoT 裝置深度整合
5. 會計總帳系統完整取代
6. 對外開放的 SaaS 商品化包裝與多品牌銷售後台

---

3. 容易被誤解的假設（Assumptions & Misreads）

1. **客戶資料歸平台所有**：加盟店是使用者，不是資料最終擁有者。
2. **VIP 歸總部管理**：店家可以服務 VIP，但 VIP 資產不屬於單店。
3. **系統先做控制權核心，再做體驗加值**：先保住資料、交易、分潤，再談行銷花樣。
4. **多層級分潤是規則引擎，不是報表函式**：不能只做靜態計算欄位。
5. **Phase 1 不是縮水版，而是核心防火牆版**：先讓旗艦店跑在自己的資料與金流上。

---

4. 建議先不做（Not Now）

1. **自動派工演算法先不做太深**：先人工/半自動，避免規則未成熟就過度自動化。
2. **行銷自動化先不做複雜旅程**：先保留名單與事件資料，之後再做自動化。

3. **課程系統先做管理，不先做完整 LMS**：先有報名、排程、成效追蹤即可。
4. **多品牌 SaaS 化先不做商業化包裝**：先把單一集團多層級架構打穩。

---

5. 使用者角色與需求

5.1 角色清單

1. 總部管理員（HQ Admin）
2. 賦能平台管理者（Enablement Admin）
3. 經銷商管理者（Distributor Admin）
4. 加盟店店主管（Store Manager）
5. 服務人員 / 技術老師 / 教練（Staff / Coach）
6. 顧客（Customer）
7. VIP 會員（VIP Member）
8. 財務/客服/審核人員（Backoffice Operator）

5.2 角色目標

- 總部：掌握數據、營收、分潤、加盟、VIP、合約、決策報表
- 經銷商：掌握進貨、合作狀態、旗下店家、產品流向
- 店家：管理預約、排班、服務履約、店內客戶經營
- 服務人員：接單、排班、查客戶摘要、完成回報、看業績
- 顧客：預約、付款、追蹤、評價、再次預約
- VIP：享受專屬方案、價格與跨店服務權益

5.3 User Stories（摘要）

總部

- 作為總部，我要能看到所有交易、分潤、VIP、加盟資料，才能控制整體生意。
- 作為總部，我要能把客戶重新分派給不同店家或服務人員，才能維持平台主導權。
- 作為總部，我要能查詢介紹鏈、合約、分潤明細，才能避免爭議。

店家

- 作為店家，我要能管理自己的預約、排班與服務紀錄，才能日常營運。
- 作為店家，我要能看到自己的營收與服務人員表現，但不能看到其他店資料。

服務人員

- 作為服務人員，我要能接單、看排班、做服務回報，才能完成到府服務。
- 作為服務人員，我只能看到必要的客戶摘要，避免過度取得個資。

顧客/VIP

- 作為顧客，我要能快速預約、付款與查詢歷史服務。
- 作為 VIP，我要能看到我的專屬價格、權益與推薦紀錄。

5.4 BDD Outline

- feature: 帳號與多層級權限
  scenarios:
  - name: 總部查看全集團資料
    summary: 總部可跨層級查詢與彙總資料
  - name: 加盟店只能看自己資料
    summary: 店家只能看到自己授權範圍內資料
- feature: 客戶主權與來源追蹤
  scenarios:
  - name: 建立全域客戶檔案
    summary: 同一顧客在不同店仍被辨識為同一主體
  - name: 平台重派客戶
    summary: 總部可依規則重新分配客戶與服務機會
- feature: 預約到履約閉環
  scenarios:
  - name: 顧客完成預約與付款
    summary: 預約、付款、派工、回報形成單一路徑
  - name: 服務人員完成回報
    summary: 服務完成後留存可稽核紀錄
- feature: 多層分潤
  scenarios:

    - name: 技術老師服務分潤
      summary: 依規則自動計算應得分潤
    - name: VIP 介紹分潤
      summary: 依介紹鏈與比例計算獎勵
- feature: VIP 控制系統
  scenarios:
    - name: VIP 跨店服務
      summary: VIP 可在授權店家間使用權益
    - name: VIP 權益與價格控管
      summary: VIP 方案、價格、權益由總部統一定義

---

6. 系統架構（System Architecture）

6.1 架構原則

1. **平台優先**：資料主權與流程控制由平台掌握。
2. **分層清楚**：前台、服務、資料、權限、報表分離。
3. **先小後大**：Phase 1 先跑核心閉環，Phase 2 商模引擎，Phase 3 規模化。
4. **隔離但可彙總**：加盟商/店家資料隔離，但總部能彙整分析。
5. **規則外部化**：分潤、權益、派工規則避免寫死在單一程式碼中。

6.2 分層架構

Client Layer

- 管理後台 Web
- 顧客端 PWA
- 服務人員端 PWA

API Layer

- BFF / API Gateway
- Internal API
- Partner/Finance API
- Webhook Receiver

Service Layer

- IAM / Auth Service
- Tenant & Org Service
- Customer Ownership Service
- Booking Service
- Scheduling / Dispatch Service
- Service Record Service
- Payment Service
- Commission Engine
- VIP Service
- Inventory Service
- Contract Service
- Reporting Service
- Notification Service
- Course & Coaching Service

Data Layer

- OLTP 關聯式資料庫
- Redis Cache
- Object Storage（圖片、附件、報表匯出）
- Audit Log Store
- Analytics Warehouse（Phase 3）

Infra Layer

- Cloud VM / Container Runtime
- Load Balancer
- WAF
- CI/CD

- Backup & Monitoring
- Secret / KMS

6.3 架構圖（文字描述版）

顧客端 PWA、服務人員端 PWA、管理後台 Web 先進入 API Gateway；
API Gateway 根據登入身份與 scope 導向對應服務；
核心服務以 Booking、Customer、Payment、Commission、VIP、Inventory、Contract 為中心；
所有交易與主資料寫入主資料庫；
高頻查詢走快取；
支付狀態、通知、對帳、報表透過事件或佇列非同步處理；
總部報表與分析自交易資料彙整到分析層；
稽核與安全事件寫入不可變更的 Audit Store。

6.4 系統互動流程

A. 顧客預約流程

1. 顧客登入/註冊
2. 選擇服務、日期、地點、時段
3. 系統檢查店家/人員可服務範圍與可用時段
4. 建立訂單草稿
5. 導向金流付款
6. 付款成功回寫
7. 系統或後台進行指派
8. 服務人員接單
9. 完成服務與回報
10. 訂單結案並更新營收、分潤、客戶價值

B. VIP / 介紹分潤流程

1. 建立 VIP 方案
2. 綁定介紹人鏈與方案權益
3. VIP 成交或服務消費
4. 系統計算介紹鏈分潤與服務分潤

5. 分潤入帳至待結算帳
6. 財務審核後出具結算批次

C. 產品進銷存流程

1. 總部建立品牌/品項/價格
2. 經銷商進貨
3. 店家領貨或出貨
4. 產生庫存異動與差價
5. 報表統計毛利與流向

6.5 模組拆分（Domain → Module）

- Identity Domain → 帳號、權限、組織、租戶
- Customer Domain → 客戶主檔、標籤、來源、關係鏈
- Commerce Domain → 服務項目、訂單、付款、退款
- Operation Domain → 排班、派工、服務紀錄、通知
- Finance Domain → 分潤、結算、對帳、報表
- Channel Domain → 經銷商、加盟商、合約
- Growth Domain → VIP、課程、行銷、評價
- Governance Domain → 稽核、安全、資料匯出控制

---

7. 控制層架構（Control Layer）

7.1 Customer Ownership Engine

目的：讓客戶是平台資產，不是單店私有資產。

核心能力：

- 全域 Customer ID
- 客戶來源追蹤
- 客戶主歸屬 = Platform
- 可指派店家 / 可指派服務人員

- 不可由店家完整匯出
- 客戶合併與去重

關鍵規則：

- 同一手機/Email/身份特徵需嘗試合併為單一客戶主體
- 店家只能看到授權的客戶視圖
- 總部可跨店查看客戶全歷程
- 客戶轉派需留下完整 audit log

7.2 Traffic Allocation Engine

目的：讓平台掌握流量與訂單分配，而不是單店完全自有。

核心能力：

- 手動派單
- 半自動建議派單
- 自動派單（Phase 3）
- 流量傾斜規則
- 黑名單 / 降權
- 新店扶持規則

預設權重例：

- 地區可服務：30%
- 時段可服務：20%
- 專長匹配：20%
- 評價表現：10%
- 響應速度：10%
- 平台策略（新店扶持/VIP 優先）：10%

7.3 VIP Control Engine

目的：讓 VIP 是總部持有、可跨店營運的高價值資產。

核心能力：

- VIP 方案管理
- VIP 等級與權益
- VIP 介紹鏈
- VIP 專屬價格
- VIP 跨店服務
- VIP LTV / 留存分析
- VIP 導單與服務池

關鍵規則：

- VIP owner = HQ
- 店家只能服務，不擁有 VIP 主權
- VIP 方案與價格由總部統一定義
- 介紹鏈變更需經審批與稽核

7.4 Conversion Funnel Engine

目的：把課程/顧問/SSA 類活動從內容模組，升級成轉化引擎。

核心能力：

- 線索 → 課程 → 會員 → VIP → 經銷/加盟 的轉化追蹤
- 活動報名與出席
- 學習/陪跑紀錄
- 成交來源歸因
- 課程營收與分潤
- 漏斗分析

---

8. 資料庫架構（Database Architecture）

8.1 多租戶/多層級策略

建議：**Row-level isolation + org hierarchy**

原因：

- 前期只有單一集團品牌，但需要多層級資料隔離
- 未來可能擴展為多品牌或 SaaS，需保留升級空間
- 以 org_id, org_type, hq_id, tenant_id 建立可彙總結構

建議欄位：

- tenant_id：品牌/集團級保留欄位
- org_id：當前組織節點
- org_type：HQ / Enablement / Distributor / Store
- owner_org_id：資料主要擁有方
- assigned_org_id：目前營運服務方

索引策略：

- 所有主交易表都需 (tenant_id, org_id, created_at) 複合索引
- 客戶表需 (tenant_id, normalized_phone)、(tenant_id, email) 去重索引
- 訂單表需 (tenant_id, assigned_org_id, status, service_date)

8.2 主要實體（Entities）

- Tenant
- Organization
- OrgRelation
- User
- Role
- Permission
- UserOrgMembership
- Customer
- CustomerAddress
- CustomerTag
- CustomerAssignment
- LeadSource

- VIPMember
- VIPPlan
- VIPReferralChain
- ServiceCategory
- ServiceItem
- BookingOrder
- BookingAssignment
- StaffProfile
- StaffSchedule
- StaffSkill
- ServiceRecord
- Payment
- Refund
- CommissionRule
- CommissionLedger
- SettlementBatch
- Distributor
- FranchiseStore
- Contract
- ProductBrand
- ProductItem
- InventoryLot
- InventoryMovement
- Course
- CourseEnrollment
- CoachingSession

- NotificationLog
- Review
- AuditLog
- ExportRequest

8.3 ERD（文字描述）

- Tenant 1-N Organization
- Organization 1-N UserOrgMembership N-1 User
- Customer 1-N BookingOrder
- BookingOrder 1-1 Payment（可拆成 1-N 交易紀錄）
- BookingOrder 1-N BookingAssignment
- BookingOrder 1-1 ServiceRecord
- VIPMember 1-N BookingOrder
- VIPMember 1-N VIPReferralChain
- CommissionRule 1-N CommissionLedger
- SettlementBatch 1-N CommissionLedger
- Organization 1-N Contract
- ProductItem 1-N InventoryMovement
- Course 1-N CourseEnrollment
- User / Organization / BookingOrder / Payment 均可關聯 AuditLog

8.4 主要資料表草圖

customer

- id (PK)
- tenant_id
- global_customer_code
- owner_org_id
- assigned_org_id

- first_touch_org_id
- source_channel
- name
- phone
- email
- birthday
- gender
- vip_member_id (nullable)
- status
- total_spent
- lifetime_value
- last_service_at
- created_at / updated_at

索引：

- unique(tenant_id, global_customer_code)
- unique(tenant_id, phone) where phone is not null
- index(tenant_id, assigned_org_id)
- index(tenant_id, vip_member_id)

booking_order

- id (PK)
- tenant_id
- order_no
- customer_id (FK)
- org_id
- assigned_org_id
- service_item_id

- schedule_start_at
- schedule_end_at
- address_id
- status
- traffic_source
- assigned_mode (manual/system)
- total_amount
- payable_amount
- paid_amount
- payment_status
- cancel_policy_snapshot
- created_at / updated_at

索引：

- unique(tenant_id, order_no)
- index(tenant_id, customer_id, created_at desc)
- index(tenant_id, assigned_org_id, status, schedule_start_at)

payment

- id (PK)
- tenant_id
- order_id (FK)
- payment_no
- gateway
- gateway_trade_no
- amount
- fee_amount
- currency

- status
- paid_at
- callback_payload_hash
- created_at / updated_at

索引：

- unique(gateway, gateway_trade_no)
- index(tenant_id, order_id)
- index(tenant_id, status, paid_at)

commission_ledger

- id (PK)
- tenant_id
- source_type
- source_id
- beneficiary_org_id
- beneficiary_user_id
- commission_rule_id
- commission_type
- rate
- amount
- status (pending/approved/settled/reversed)
- settlement_batch_id
- created_at / updated_at

索引：

- index(tenant_id, beneficiary_org_id, status)
- index(tenant_id, beneficiary_user_id, status)
- index(tenant_id, settlement_batch_id)

vip_member

- id (PK)
- tenant_id
- customer_id
- owner_org_id
- vip_plan_id
- level
- referral_root_id
- exclusive_price_group_id
- valid_from
- valid_to
- status
- created_at / updated_at

索引：

- unique(tenant_id, customer_id)
- index(tenant_id, owner_org_id, status)

contract

- id (PK)
- tenant_id
- org_id
- contract_type
- contract_no
- start_date
- end_date
- status
- auto_renew

- attachment_path
- created_at / updated_at

索引：

- unique(tenant_id, contract_no)
- index(tenant_id, org_id, status, end_date)

8.5 資料治理要求

- PII 欄位（電話、地址、身份資料、銀行資訊）需加密或至少欄位遮罩
- 刪除採 soft delete + legal hold，不做實體刪除
- 關鍵主檔變更需版本化或寫入 audit log
- 匯出資料需申請流程與審核紀錄

---

9. API Architecture

9.1 API 分層

- **Internal API**：前後台、PWA 使用
- **Partner API**：金流、未來外部合作方
- **Admin API**：總部專用操作，如重派客戶、調整規則
- **Webhook API**：支付回調、通知回傳

9.2 主要 Endpoint 大綱

Auth / IAM

- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- POST /auth/password/reset
- GET /me

Organizations

- GET /orgs

- GET /orgs/{id}
- POST /orgs
- PATCH /orgs/{id}
- GET /orgs/{id}/users

Customers

- GET /customers
- POST /customers
- GET /customers/{id}
- PATCH /customers/{id}
- POST /customers/merge
- POST /customers/{id}/assign
- GET /customers/{id}/journey
- POST /customers/export-requests

Booking / Service

- GET /service-items
- POST /bookings
- GET /bookings/{id}
- PATCH /bookings/{id}/cancel
- PATCH /bookings/{id}/reschedule
- POST /bookings/{id}/assign
- POST /bookings/{id}/accept
- POST /bookings/{id}/complete
- GET /bookings/{id}/service-record

Staff / Schedule

- GET /staff
- GET /staff/{id}/availability

- POST /staff/{id}/schedules
- POST /dispatch/suggestions

Payments

- POST /payments/create
- POST /payments/webhooks/newebpay
- GET /payments/{id}
- POST /refunds
- GET /reconciliation/batches

Commission / Settlement

- GET /commission-rules
- POST /commission-rules
- GET /commission-ledgers
- POST /settlements/batches
- POST /settlements/{id}/approve

VIP

- GET /vip/plans
- POST /vip/plans
- POST /vip/members
- GET /vip/members/{id}
- POST /vip/referrals
- GET /vip/{id}/benefits

Inventory

- GET /products
- POST /products
- POST /inventory/inbound
- POST /inventory/outbound

- GET /inventory/movements

Contract

- GET /contracts
- POST /contracts
- PATCH /contracts/{id}
- GET /contracts/expiring

Course & Coaching

- GET /courses
- POST /courses
- POST /courses/{id}/enrollments
- POST /coaching-sessions
- GET /funnel/reports

9.3 Webhook 設計

- POST /webhooks/newebpay/payment
- POST /webhooks/newebpay/refund
- POST /webhooks/notification/delivery

Webhook 原則：

- 驗簽
- 冪等處理
- 原始 payload 保留 hash
- 失敗重試
- 進入 dead-letter queue 做人工處理

9.4 API Key 設計

- 不儲存明文，只存 hash
- Scopes：read / write / finance / admin
- Rate limit：

    - key：每秒 10、每分鐘 300
    - tenant：每秒 50、每分鐘 3000
    - IP：每秒 20、每分鐘 600
- 可選 IP 白名單
- Key rotation：90 天有效期，支援提前輪替與吊銷
- Audit log：記錄 key 建立、使用、失敗、權限變更、停用
- 風險控管：WAF、異常偵測、頻繁失敗封鎖、ASN/IP 風險分級

---

10. AI 模組（本版先做低風險導入）

10.1 建議 AI Use Cases

1. 服務紀錄摘要：把老師輸入的長備註整理成結構化摘要
2. 客戶標籤建議：依歷史服務與偏好給建議標籤
3. 客服問答草稿：回覆常見預約/方案/改期問題
4. 派工建議：先做推薦，不直接自動執行

10.2 本階段不建議

- 直接用 AI 自動做最終派工決策
- 用 AI 自動決定分潤
- 用 AI 直接修改 VIP 或財務資料

10.3 AI 安全性

- 敏感個資先遮罩後再送模型
- 每次 AI 建議需標示為 suggestion，不直接落資料庫
- 不可跨 tenant / 組織混用資料
- 高風險操作需二次確認

---

11. 系統安全性（Security Architecture）

11.1 AuthN

- 員工端 / 後台：Email/手機 + 密碼 + MFA
- 顧客端：手機 OTP / Email OTP / 密碼式登入
- 管理端可擴充 SSO（Google Workspace / Microsoft）

11.2 AuthZ

採 **RBAC + Resource Ownership + Data Scope**

角色只是第一層；真正授權判斷還要看：

- 角色（HQ / Distributor / Store / Staff）
- 所屬組織
- 資料 owner_org_id
- assigned_org_id
- 特殊 scope（finance / export / audit）

11.3 加密

- 全站 TLS 1.3
- 敏感資料 at-rest 加密（AES-256）
- 金鑰由 KMS 管理
- 資料庫備份加密

11.4 API 風控

- WAF
- Rate limiting
- Bot protection（顧客端）
- 高風險操作 re-auth
- 管理端異地登入告警

11.5 稽核

需保留不可竄改或 append-only log：

- 客戶重派
- VIP 方案建立/變更

- 分潤規則調整
- 合約異動
- 退款
- 匯出申請
- 權限變更
- API key 建立/吊銷

11.6 日誌管理

- PII 遮蔽
- 保留期分級：交易/稽核至少 1–3 年，依法務要求調整
- 重要安全事件進告警系統

11.7 災難備援

- RPO：1 小時內（核心交易資料）
- RTO：4 小時內（核心營運）
- 每日備份 + 週期性還原演練
- 關鍵支付回調與分潤資料需可重放/重算

---

12. 非功能需求（NFR）

12.1 效能

- 後台查詢：P95 < 800ms
- 顧客預約頁：P95 < 1.2s
- 支付回調處理：一般 5 秒內完成狀態落地
- 核心交易系統支援初期 20–50 QPS，保留水平擴展能力

12.2 擴充性

- 前後端分離
- 核心規則模組化
- 長報表與通知非同步化

- 讀多寫少場景使用 Redis 快取
- 報表/分析未來可拆到 warehouse

12.3 高可用

- 單區高可用起步，保留跨區備援能力
- 支付、訂單、客戶主檔為最優先保護資產

12.4 故障復原

- Payment callback 冪等
- 任務重試
- 佇列死信箱
- 退款與分潤支援人工補單/沖正

12.5 成本項

- 應用伺服器
- 關聯式資料庫
- 快取
- 物件儲存
- 監控與日誌
- 推播/SMS
- AI 調用（如啟用）

---

13. Phase 規劃與交付邊界

13.1 Phase 1｜核心防火牆

目標：旗艦店開業即可跑在自己的系統與金流上。

範圍：

- 多角色帳號權限
- 服務項目管理
- CRM 客戶管理（含來源追蹤）

- 服務紀錄與備註
- 預約管理（手動指派）
- 基本排班
- 金流串接
- 基本報表
- 顧客端 PWA
- 服務人員端 PWA
- **必補控制權基礎**：Global Customer ID、客戶不可匯出、總部可重派、VIP 欄位預留

驗收口徑：

- 預約 → 付款 → 指派 → 服務 → 回報 → 入帳 跑通
- 旗艦店正式可用
- 客戶資料統一進平台主檔

13.2 Phase 2｜商模引擎

目標：把最核心的錢與關係數位化。

範圍：

- 分潤引擎
- 產品進銷存
- VIP 方案管理
- 經銷商管理
- 合約管理
- VIP 介紹鏈
- 分潤待結算帳
- 基本結算批次

驗收口徑：

- 技術老師分潤、VIP 介紹分潤跑通
- 產品差價與進銷存可查

- VIP 方案可追蹤
- 可對外展示完整商模 Demo

13.3 Phase 3｜規模化擴展

目標：支撐加盟擴張與跨區複製。

範圍：

- 加盟商獨立後台
- 多據點分析
- 課程與教練系統
- 行銷工具
- 評價系統
- 退款與改期規則
- 自動派工建議/演算法
- 進階報表與儀表板

驗收口徑：

- 多據點獨立運作
- 課程與行銷模組可用
- 報表與派工建議支援規模化營運

---

14. 風險 × 限制 × 建議

14.1 技術風險

- 客戶去重若設計不好，後續會有大量重複主檔
- 分潤規則若直接寫死，後面會難維護
- 多層級權限若只做 RBAC，資料外洩風險高
- 退款、取消、改期若沒先定清楚，會直接影響帳務

14.2 營運風險

- 店家對「客戶資料主權在平台」可能有心理阻力

- VIP 歸總部會涉及利益分配說明
- 若沒有標準作業流程，系統再好也會被人工繞過

14.3 團隊限制

- 若沒有產品 owner 持續定義規則，開發易反覆
- 若財務規則晚決定，Phase 2 容易重工

14.4 建議

1. 先確立「哪些資料一定歸總部」
2. 先確立「分潤與退款規則表」
3. 先確立「VIP 是否跨店、如何重派、誰可看」
4. Phase 1 先把控制權地基打穩，不要只追求畫面多

---

15. 業主討論版：本次應確認的決策題

1. 客戶資料是否 100% 歸總部，加盟店僅有使用權？
2. VIP 是否明確歸總部，而非單店？
3. 總部是否可以重派顧客與服務機會？
4. 分潤規則是否先固定一版，再做規則引擎？
5. Phase 1 是否接受先人工派單、Phase 3 再做自動派工？
6. 產品進銷存是 HQ 主導、經銷商主導，還是雙軌？
7. 課程系統第一版是做報名管理，還是要直接綁轉單漏斗？
8. 店家是否可以匯出客戶資料？若可以，條件是什麼？
9. 哪些操作一定要審批：退款、重派、合約異動、VIP 鏈變更？
10. 業主最在意的是：先營運、先招商、先控客戶，還是先做完整體驗？

---

16. 最終建議（給業主的白話結論）

這套系統不要用「預約系統」的角度討論，會低估太多。

正確的角度是：

它是一套把客戶、交易、VIP、加盟、分潤都收回到總部平台的控制型系統。

所以，真正該先做的，不是功能多，而是：

- 客戶資料先進平台
- 金流先進平台
- 分潤規則先可計算
- VIP 先可被總部管理
- 店家與人員先跑在平台規則下

只要這幾個先做到，後面不論是加盟擴張、課程導流、VIP 經營、行銷自動化，才會建立在自己的地基上，而不是建立在別人的工具上。